

# 前端架构设计方案（Vue.JS 版）

撰写人：

2019 年 03 月

目录

- 1 前言 ..... 3
- 2 前后端分离架构应用场景 ..... 3
  - 2.1.1 主要应用场景 ..... 3
  - 2.1.2 术业有专攻（开发人员分离） ..... 3
- 3 我们前端的架构建议 ..... 4
  - 3.1 MVVM 架构模式 ..... 5
  - 3.2 工作流程图 ..... 5
  - 3.3 VUE.JS 前端技术选择 ..... 6
  - 3.4 VUE.JS 优缺点分析 ..... 7
    - 3.4.1 优点 ..... 7
    - 3.4.2 缺点 ..... 10
- 4 我们开发流程的建议 ..... 10
- 5 扩展内容说明 ..... 11
  - 5.1 前后端分离解释 ..... 11
  - 5.2 为什么要前后端分离 ..... 11
    - 5.2.1 不分离高度耦合架构 ..... 11
    - 5.2.2 前后端分离架构 ..... 13

# 1 前言

技术一定是要走在业务前面的，否则将错过最佳的发展期。

以下文档介绍我们对项目前端开发的建议。

## 2 前后端分离架构应用场景

### 2.1.1 主要应用场景

- 1) 针对具有复杂交互逻辑的前端应用
- 2) 提供基础的架构抽象
- 3) 通过 Ajax 数据持久化，保证前端用户体验

好处就是当前后端进行一些数据交互的时候，前端可以通过 Ajax 请求对后端做数据持久化，不需要刷新整个页面，只需要改动 DOM 里需要改动的那部分数据和内容，特别是对于移动端应用场景，刷新页面的代价太昂贵，会重新加载很多资源，虽然有些资源会被缓存，但是页面的 DOM、JS、CSS 都会被浏览器重新解析一遍，因此，移动端页面经常会做成 SPA 单页应用，在这个基础上就诞生了很多 MVVM 框架，如 AngularJS、ReactJS、Vue.js。

### 2.1.2 术业有专攻（开发人员分离）

随着时代的发展，渐渐的许多公司开始把前后端的界限分的越来越明确，前端工程师只管前端的事情，后端工程师只管后端的事情。正所谓术业有专攻，能够为平台各个环节优质的服务以及产出。

### 2.1.2.1 后端 Java、PHP 开发

把精力放在基础，设计模式，jvm 原理，解释器原理，spring+springmvc  
Yii、ThinkPHP、Yaf、Laravel、CodeIgniter 等框架原理及源码，linux，  
mysql 事务隔离与锁机制，mongodb，http/tcp，多线程，分布式架构，弹性计  
算架构，微服务架构，java 性能优化，以及相关的项目管理等等。

后端追求的是：三高（高并发，高可用，高性能），安全，存储，业务等  
等。

### 2.1.2.2 前端开发

把精力放在 HTML，CSS，Jquery，AngularJS，Bootstrap，ReactJS，  
Vue.js，Webpack，LESS/SASS，GULP，NodeJS，Google V8 引擎，JavaScript  
多线程，模块化，面向切面编程，设计模式，浏览器兼容性，性能优化等等。

前端追求的是：页面表现，速度流畅，兼容性，用户体验等等。

## 3 我们前端的架构建议

我们使用的是前后端分离的前端架构模式，这也是目前互联网项目最欢迎  
的设计系统架构模式。

前后端分离已成为互联网项目开发的业界标准使用方式，通过 Nginx +  
Tomcat 的方式（也可以中间加一个 NodeJS）有效的进行解耦，并且前后端分离  
会为以后的大型分布式架构、弹性计算架构、微服务架构、多端化服务（多种  
客户端，例如：浏览器，车载终端，安卓，IOS 等等）打下坚实的基础。这个  
步骤是系统架构从猿进化成人的必经之路。

核心思想是前端 HTML 页面通过 Ajax 调用后端的 RESTful API 接口并使用 JSON 数据进行交互。

### 3.1 MVVM 架构模式

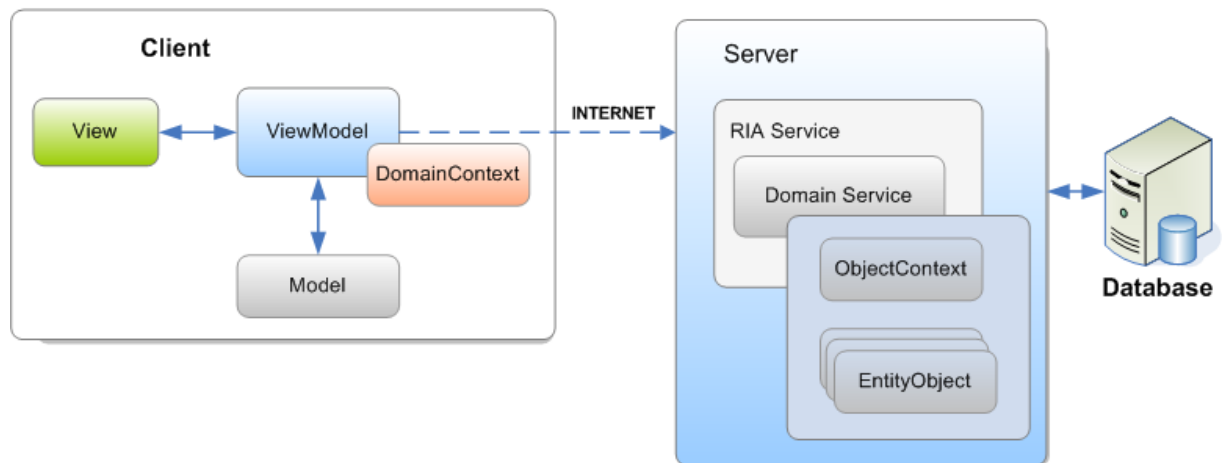
MVVM 是 Model-View-ViewModel 的缩写；

MVVM 模式和 MVC 模式一样，主要目的是分离视图（View）和模型（Model），有几大好处

1. **低耦合。**视图（View）可以独立于 Model 变化和修改，一个 ViewModel 可以绑定到不同的“View”上，当 View 变化的时候 Model 不可以不变，当 Model 变化的时候 View 也可以不变。
2. **可重用性。**你可以把一些视图逻辑放在一个 ViewModel 里面，让很多 view 重用这段视图逻辑。
3. **独立开发。**开发人员可以专注于业务逻辑和数据的开发（ViewModel），设计人员可以专注于页面设计，使用 Expression Blend 可以很容易设计界面并生成 xaml 代码。
4. **可测试。**界面素来是比较难于测试的，而现在测试可以针对 ViewModel 来写。

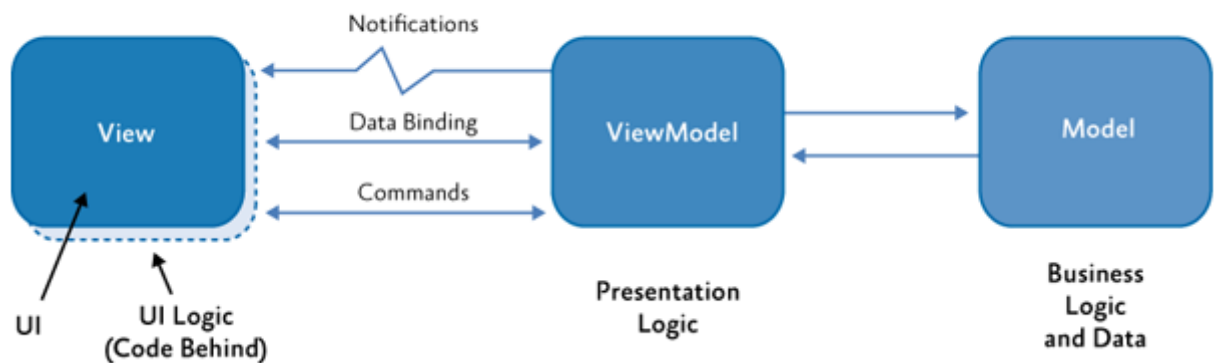
### 3.2 工作流程图

- 1)、完整开发流程



## 2)、MVVM 开发模式

The MVVM classes and their interactions



## 3.3 Vue. JS 前端技术选择

前端 MVVM 技术框架有很多，如 AngularJS、ReactJS、Vue. js 等等；

不同的技术框架都存在不同优缺点，综合大部分情况下，我们推荐使用 Vue. JS 进行前端技术开发。

Vue. js 是一套用于构建用户界面的渐进式框架。与其它大型框架不同的是，Vue 被设计为可以自底向上逐层应用。Vue 的核心库只关注视图层，不仅易于上手，还便于与第三方库或既有项目整合。另一方面，当与现代化的工具链以及各种支持类库结合使用时，Vue 也完全能够为复杂的单页应用提供驱动。

Vue.js 的目标是通过尽可能简单的 API 实现响应的数据绑定和组合的视图组件。

Vue.js 的核心思想包括两个方面：数据驱动和组件化

数据驱动：DOM 是数据的一种自然映射，在 vue.js 中只需要操作数据，vue.js 通过 directives 指令去对 DOM 做一层封装，当数据发生变化，会通知指令去修改对应的 DOM，数据驱动 DOM 变化，DOM 是数据的一种自然映射，vue.js 还会对操作做一些监听，当我们修改视图的时候，vue.js 监听到这些变化，从而去改变数据，这样就实现了数据的双向绑定。

组件化的目的是扩展 HTML 元素，封装可重用的代码

组件设计原则：

- 1) 页面上每个独立的可视/可交互区域视为一个组件，
- 2) 每个组件对应一个工程目录，组件所需要的各种资源在这个目录下就近维护，
- 3) 页面不过是组件的容器，组件可以嵌套自由组合形成完整的页面。

## 3.4 Vue. JS 优缺点分析

### 3.4.1 优点

#### 3.4.1.1 高可用性

使用 Vue.js，你无需使用 MobX 或 React Router 等非官方软件包来处理应用程序的关键部分。Vue.js 提供了 Vue Router 和 Vuex。这些都是很好的库，而且是为 Vue.js 量身定制的。

### 3.4.1.2 速度

Vue.js 非常快。它也许不是最快的，但它的性能表现对绝大多数 Web 项目来说都是顶级的。你上一次需要每秒渲染和更新数千个 DOM 元素是什么时候？

### 3.4.1.3 响应性

数据绑定在前端领域是个大问题。现在我们更专注于数据，而不像使用 jQuery 一样对 DOM 进行微观管理。Vue 通过双向响应数据绑定系统巧妙地处理这个问题。

为实现这种响应性，Vue.js 为状态中的每个变量添加了许多 getter 和 setter，以便它可以跟踪更改并自动更新 DOM。这种方法并不完美，我们稍后会再提到。

### 3.4.1.4 双向数据绑定

也就是所谓的响应式数据绑定。这里的响应式不是 @media 媒体查询中的响应式布局，而是指 Vue.js 会自动对页面中某些数据的变化做出同步的响应。

也就是说，Vue.js 会自动响应数据的变化情况，并且根据用户在代码中预先写好的绑定关系，对所有绑定在一起的数据和视图内容都进行修改。而这种绑定关系，就是以 input 标签的 v-model 属性来声明的，因此你在别的地方可能也会看到有人粗略的称 Vue.js 为声明式渲染的模版引擎。

这也就是 Vue.js 最大的优点，通过 MVVM 思想实现数据的双向绑定，让开发者不用再操作 dom 对象，有更多的时间去思考业务逻辑。

### 3.4.1.5 组件化开发

在前端应用，我们是否也可以像编程一样把模块封装呢？这就引入了组件化开发的思想。

Vue.js 通过组件，把一个单页应用中的各种模块拆分到一个一个单独的组件（component）中，我们只要先在父级应用中写好各种组件标签（占坑），并且在组件标签中写好要传入组件的参数（就像给函数传入参数一样，这个参数叫做组件的属性），然后再分别写好各种组件的实现（填坑），然后整个应用就算做完了。

### 3.4.1.6 Virtual DOM

现在的网速越来越快了，很多人家里都是几十甚至上百 M 的光纤，手机也是 4G 起步了，按道理一个网页才几百 K，而且浏览器本身还会缓存很多资源文件，那么几十 M 的光纤为什么打开一个之前已经打开过，已经有缓存的页面还是感觉很慢呢？这就是因为浏览器本身处理 DOM 也是有性能瓶颈的，尤其是在传统开发中，用 JQuery 或者原生的 JavaScript DOM 操作函数对 DOM 进行频繁操作的时候，浏览器要不停的渲染新的 DOM 树，导致页面看起来非常卡顿。

而 Virtual DOM 则是虚拟 DOM 的英文，简单来说，他就是一种可以预先通过 JavaScript 进行各种计算，把最终的 DOM 操作计算出来并优化，由于这个 DOM 操作属于预处理操作，并没有真实的操作 DOM，所以叫做虚拟 DOM。最后在计算完毕才真正将 DOM 操作提交，将 DOM 操作变化反映到 DOM 树上。

### 3.4.1.7 轻量高效

Vue.js 通过简洁的 API 提供高效的数据绑定和灵活的组件系统

### 3.4.1.8 动画系统

Vue.js 提供了简单却强大的动画系统，当一个元素的可见性变化时，用户不仅可以很简单地定义对应的 CSS Transition 或 Animation 效果，还可以利用丰富的 JavaScript 钩子函数进行更底层的动画处理。

### 3.4.2 缺点

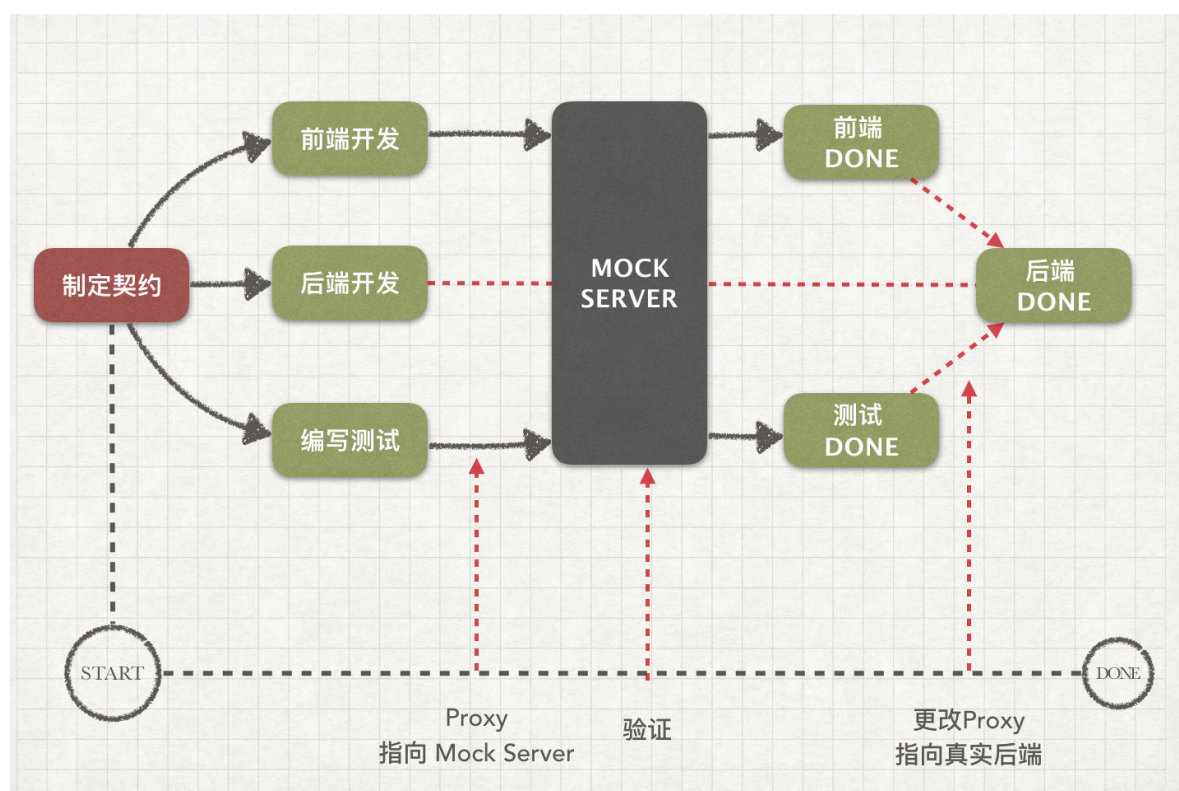
1)、Vue.js 不缺入门教程，可是很缺乏高阶教程与文档。同样的还有书籍

2)、Vue.js 不支持低版本 IE 浏览器，如 IE8 及以下版本

3)、模板的运行时错误描述不够直观，这个跟 Angular 1 有点类似。

Vue.js 为 JS 代码提供了很多有用的警告信息，例如当你试图改变 prop 或不恰当地使用 data() 方法时，它会给出警告。这也是从 React 借鉴过来的比较好的方面。但对模板的运行时错误处理仍然是 Vue 的一个弱项，它的异常堆栈信息总是指向 Vue.js 的内部方法，不够直观。

## 4 我们开发流程的建议



## 5 扩展内容说明

### 5.1 前后端分离解释

前后端分离已成为互联网项目开发的业界标准使用方式，通过 Nginx+Tomcat 的方式（也可以中间加一个 NodeJS）有效的进行解耦，并且前后端分离会为以后的大型分布式架构、弹性计算架构、微服务架构、多端化服务（多种客户端，例如：浏览器，车载终端，安卓，IOS 等等）打下坚实的基础。这个步骤是系统架构从猿进化成人的必经之路。

核心思想是前端 HTML 页面通过 Ajax 调用后端的 RESTful API 接口并使用 JSON 数据进行交互。

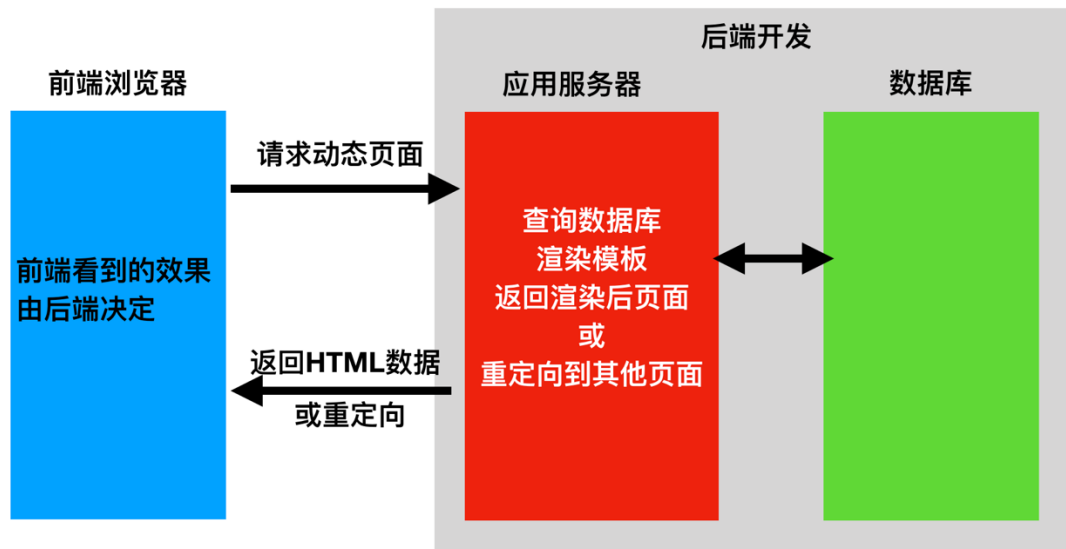
### 5.2 为什么要前后端分离

#### 5.2.1 不分离高度耦合架构

##### 5.2.1.1 架构概况

在前后端不分离的应用模式中，前端页面看到的效果都是由后端控制，由后端渲染页面或重定向，也就是后端需要控制前端的展示，前端与后端的耦合度很高。

这种应用模式比较适合纯网页应用，但是当后端对接 App 时，App 可能并不需要后端返回一个 HTML 网页，而仅仅是数据本身，所以后端原本返回网页的接口不再适用于前端 App 应用，为了对接 App 后端还需再开发一套接口。



### 5.2.1.2 技术分析

以前开发模式的 JavaWeb、PHP、C#项目都是使用了若干后台框架，Spring/Struts、Yii、Dot NET、ThinkPHP、CodeIgniter 等等。

大多数项目在 java 后端都是分了三层，控制层，业务层，持久层。控制层负责接收参数，调用相关业务层，封装数据，以及路由&渲染到 JSP、PHP 页面。然后 JSP、PHP 页面上使用各种标签或者手写表达式将后台的数据展现出来，玩的是 MVC 那套思路。

项目需求定完了，代码写完了，测试测完了，需要工具把代码打成一个包，然后把这个包发布到生产环境下的 Web 容器里。发布完了之后前后端代码是不是全都在那个包里。包括你的 JS，CSS，图片，各种第三方的库。

当有 100 张图片，此时，用户的看似一次 http 请求，其实并不是一次，用户在第一次访问的时候，浏览器中不会有缓存，100 张图片浏览器要连着请求 100 次 http 请求，服务器接收这些请求，都需要耗费内存去创建 socket 来玩 tcp 传输。

这样的话，服务器的压力会非常大，因为页面中的所有请求都是只请求到这台服务器上，如果 1 个人还好，如果 10000 个人并发访问呢，那服务器能扛住多少个 tcp 连接？带宽有多大？服务器的内存有多大？硬盘是高性能的吗？你能抗住多少 IO？web 服务器分的内存有多大？会不会宕机？

这就是为什么，越是大中型的 web 应用，越是要解耦。理论上可以把你的数据库+应用服务+消息队列+缓存+用户上传的文件+日志+等等都扔在一台服务器上，也不用什么服务治理，也不用做什么性能监控，什么报警机制等等，就乱成一锅粥好了。但是这样就好像是把鸡蛋都放在一个篮子里，隐患非常大。如果因为一个子应用的内存不稳定导致整个服务器内存溢出而 hung 住，那整个网站就挂掉了。

如果出意外挂掉，而恰好这时业务又处于井喷式发展高峰期，那么业务成功被技术卡住，很可能会流失大量用户，后果不堪设想。

正常的互联网架构，是都要拆开的，web 服务器集群，你的应用服务器集群+文件服务器集群+数据库服务器集群+消息队列集群+缓存集群等等。

## 5.2.2 前后端分离架构

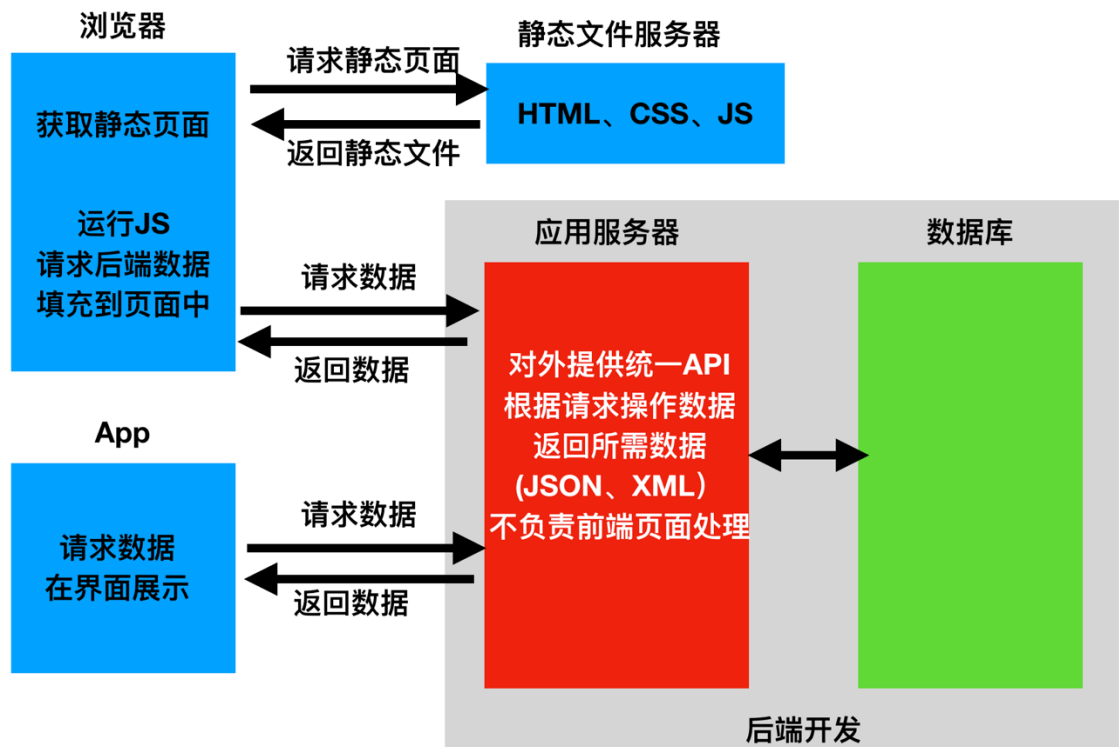
### 5.2.2.1 架构概况

在前后端分离的应用模式中，后端仅返回前端所需的数据，不再渲染 HTML 页面，不再控制前端的效果。至于前端用户看到什么效果，从后端请求的数据如何加载到前端中，都由前端自己决定，网页有网页的处理方式，App 有 App 的处理方式，但无论哪种前端，所需的数据基本相同，后端仅需开发一套逻辑对外提供数据即可。

在前后端分离的应用模式中，前端与后端的耦合度相对较低。

在前后端分离的应用模式中，我们通常将后端开发的每个视图都称为一个接口，或者 API，前端通过访问接口来对数据进行增删改查。

对应的数据交互如下图：



### 5.2.2.2 技术分析

1、可以实现真正的前后端解耦，前端服务器使用 nginx。前端/WEB 服务器放的是 css, js, 图片等等一系列静态资源（甚至你还可以 css, js, 图片等资源放到特定的文件服务器，例如阿里云的 oss，并使用 cdn 加速），前端服务器负责控制页面引用&跳转&路由，前端页面异步调用后端的接口，后端/应用服务器使用 Tomcat（把 tomcat 想象成一个数据提供者），加快整体响应速度。（这里需要使用一些前端工程化的框架比如 nodejs, react, router, react, redux, webpack）

2、发现 bug，可以快速定位是谁的问题，不会出现互相踢皮球的现象。页面逻辑，跳转错误，浏览器兼容性问题，脚本错误，页面样式等问题，全部由前端工程师来负责。接口数据出错，数据没有提交成功，应答超时等问题，全部由后端工程师来解决。双方互不干扰，前端与后端是相亲相爱的一家人。

3、在大并发情况下，同时水平扩展前后端服务器，比如淘宝的一个首页就需要 2000+ 台前端服务器做集群来抗住日均多少亿+的日均 pv。

4、减少后端服务器的并发/负载压力。除了接口以外的其他所有 http 请求全部转移到前端 nginx 上，接口的请求调用 tomcat，参考 nginx 反向代理 tomcat。且除了第一次页面请求外，浏览器会大量调用本地缓存。

5、即使后端服务暂时超时或者宕机了，前端页面也会正常访问，只不过数据刷不出来而已。

6、需要有微信相关的轻应用，那样接口完全可以共用，如果也有 app 相关的服务，那么只要通过一些代码重构，也可以大量复用接口，提升效率。

（多端应用）

7、页面显示的东西再多也不怕，因为是异步加载。

8、nginx 支持页面热部署，不用重启服务器，前端升级更无缝。

9、增加代码的维护性&易读性（前后端耦合在一起的代码读起来相当费劲）。

10、提升开发效率，因为可以前后端并行开发，而不是像以前的强依赖。

11、在 nginx 中部署证书，外网使用 https 访问，并且只开放 443 和 80 端口，其他端口一律关闭（防止黑客端口扫描），内网使用 http，性能和安全都有保障。

12、前端大量的组件代码得以复用，组件化，提升开发效率，抽出来！